Documentation technique

Outil GRC

Suivi de la conformité ISO/IEC 27001:2022 & DNSSI
et registre des risques

Application web Django — cas d'étude : *AtlasPay*, fintech marocaine fictive

Douae Admi
Élève ingénieure en cybersécurité
github.com/Dorvae

Python · Django 6.0 · SQLite

Version 1.2 — novembre 2026

Table des matières

1 Synthèse **2**

2 Contexte et objectifs **2**
2.1 Contexte . . . 2
2.2 Objectifs . . . 2
2.3 Périmètre du cas d'étude . . . 3

3 Référentiels et catalogue de contrôles **3**

4 Architecture de l'application **3**

5 Modèle de données **4**

6 Logique métier **5**
6.1 Taux de conformité par domaine . . . 5
6.2 Cotation des risques . . . 6

7 Réalisation pas à pas **6**
7.1 Back-office d'administration . . . 6
7.2 Domaines et contrôles . . . 7
7.3 Évaluations . . . 8
7.4 Vérification dans le shell Django . . . 9
7.5 Dashboard . . . 9

8 Résultats — évaluation d'AtlasPay **11**
8.1 Conformité par domaine . . . 11
8.2 Registre des risques . . . 11
8.3 Analyse . . . 11

9 Tests et validation **12**

10 Revue de sécurité et de qualité **12**

11 Limites et feuille de route **13**

12 Guide d'installation **13**

13 Conclusion **14**

A Structure du dépôt **15**

1 Synthèse

Ce document présente la conception, la réalisation et les résultats d'une application web de **Gouvernance, Risques et Conformité (GRC)** développée avec le framework Django. L'outil tient un catalogue de contrôles de sécurité, enregistre le niveau de conformité de l'organisation pour chaque contrôle, calcule un **taux de conformité par domaine** et maintient un **registre des risques** dans lequel chaque risque est évalué et relié aux contrôles qui le traitent.

Deux référentiels sont couverts : la norme internationale **ISO/IEC 27001:2022** (annexe A, organisée en quatre thèmes) et la **DNSSI** marocaine (*Directive Nationale de la Sécurité des Systèmes d'Information*). L'outil a été appliqué à **AtlasPay**, une fintech de paiement fictive ; les évaluations ont été saisies les 27 et 28 juillet 2026.

Élément	Résultat
Référentiels	ISO/IEC 27001:2022 (4 thèmes) + DNSSI
Catalogue	5 domaines, 8 contrôles, 8 évaluations, 3 risques
Domaine le plus faible	Technologique — **0 %** (authentification et cryptographie non conformes)
Domaines les plus solides	Personnes et Physique — 50 %
Registre des risques	2 risques **critiques**, 1 risque **élevé**
Livrables	Application Django, back-office d'administration, dashboard, formulaires de saisie, tests, jeu de données de démonstration

Tab. 1: Chiffres clés

Constat principal

Chaque risque du registre est relié à au moins un contrôle non conforme ou partiellement conforme. Les deux risques critiques reposent sur des contrôles technologiques (A.8.5 Authentification sécurisée, A.8.24 Utilisation de la cryptographie) tous deux évalués *non conformes* : ce sont les premières priorités de remédiation.

2 Contexte et objectifs

2.1 Contexte

Les organisations des secteurs réglementés, comme les services de paiement, doivent démontrer que leurs contrôles de sécurité de l'information sont en place et efficaces. Ce suivi se fait souvent dans des tableurs dispersés, ce qui rend difficile la lecture du niveau global de conformité, le suivi des progrès entre deux audits, ou le lien entre un écart et le risque métier qu'il crée.

Au Maroc, les entités d'importance vitale et les infrastructures critiques sont en outre soumises à la DNSSI publiée par la DGSSI, qui ajoute à la base ISO 27001 des exigences organisationnelles (par exemple un responsable de la sécurité indépendant de l'exploitation informatique) et des exigences de souveraineté des données.

2.2 Objectifs

- **Centraliser** le catalogue de contrôles ISO 27001:2022 et DNSSI dans une base de données unique.
- **Évaluer** chaque contrôle avec un statut normalisé et conserver un enregistrement daté et commenté.

- **Mesurer** automatiquement la conformité par domaine selon une règle de pondération transparente.
- **Recenser les risques** avec un score probabilité × impact et les relier aux contrôles qui les traitent.
- **Visualiser** la posture de sécurité sur un dashboard lisible par la direction.

2.3 Périmètre du cas d'étude

AtlasPay est une fintech fictive qui fournit des API de paiement aux commerçants, exploite une partie de son infrastructure en interne (on-premise) et traite des données personnelles et de paiement de ses clients. L'évaluation porte sur un sous-ensemble représentatif de huit contrôles plutôt que sur les 93 contrôles de l'annexe A.

3 Référentiels et catalogue de contrôles

L'ISO/IEC 27001:2022 regroupe les 93 contrôles de son annexe A en quatre thèmes : organisationnel (5.x), humain (6.x), physique (7.x) et technologique (8.x). L'outil utilise ces thèmes comme *domaines* ; les exigences DNSSI forment un cinquième domaine.

Code	Contrôle	Domaine	Référentiel
A.5.9	Inventaire des informations et autres actifs associés	Organisationnel	ISO 27001
A.5.19	Sécurité de l'information dans les relations avec les fournisseurs	Organisationnel	ISO 27001
A.6.3	Sensibilisation, enseignement et formation à la sécurité	Personnes	ISO 27001
A.7.1	Périmètres de sécurité physique	Physique	ISO 27001
A.8.5	Authentification sécurisée	Technologique	ISO 27001
A.8.24	Utilisation de la cryptographie	Technologique	ISO 27001
RSSI	Organisation de la sécurité — séparation des fonctions DSI et RSSI	DNSSI	DNSSI
SOUV	Souveraineté et hébergement des données	DNSSI	DNSSI

Tab. 2: Catalogue de contrôles d'AtlasPay

4 Architecture de l'application

L'outil suit le modèle **Modèle–Vue–Template (MVT)** de Django. Une seule application, `core`, porte la logique GRC ; le paquet projet, `grc_project`, porte la configuration et le routage. Les données sont stockées dans SQLite, suffisant pour un outil mono-équipe et sans serveur de base de données.

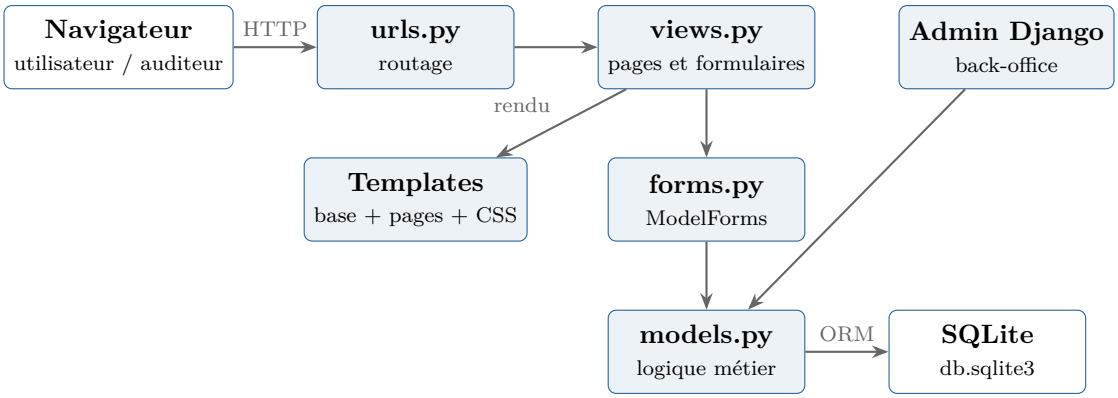


Fig. 1: Flux d'une requête et composants

Fichier	Rôle
`grc_project/settings.py`	Configuration : applications, middlewares de sécurité (CSRF, sessions, clickjacking), base de données, templates
`grc_project/urls.py`	Associe les cinq URL à l'admin et aux vues de `core`
`core/models.py`	Modèle de données et règles métier (taux de conformité, niveau de risque)
`core/views.py`	Accueil, dashboard, vues de création d'évaluations et de risques
`core/forms.py`	ModelForms générant des formulaires HTML validés
`core/admin.py`	Enregistre les quatre modèles dans l'admin Django
`core/templates/`	Gabarit `base.html` avec barre de navigation, étendu par chaque page
`core/static/core/style.css`	Code couleur des taux de conformité et des niveaux de risque

Tab. 3: Composants

URL	Vue	Fonction
`/`	`home`	Page d'accueil
`/dashboard/`	`dashboard`	Conformité par domaine + registre des risques
`/evaluations/ajouter/`	`ajouter_evaluation`	Saisir l'évaluation d'un contrôle
`/risques/ajouter/`	`ajouter_risque`	Ajouter un risque et ses contrôles
`/admin/`	Admin Django	Back-office de toutes les entités

Tab. 4: Plan des URL

Les deux vues de création suivent le modèle *Post/Redirect/Get* : un POST valide enregistre l'objet puis redirige vers le dashboard, ce qui évite les doubles soumissions lors d'un rafraîchissement. Toutes les pages étendent un gabarit commun `base.html` qui fournit la barre de navigation.

5 Modèle de données

Un **Domaine** regroupe des **Contrôles** ; chaque contrôle reçoit une ou plusieurs **Évaluations** ; un **Risque** est relié aux contrôles qui le traitent par une relation plusieurs-à-plusieurs.

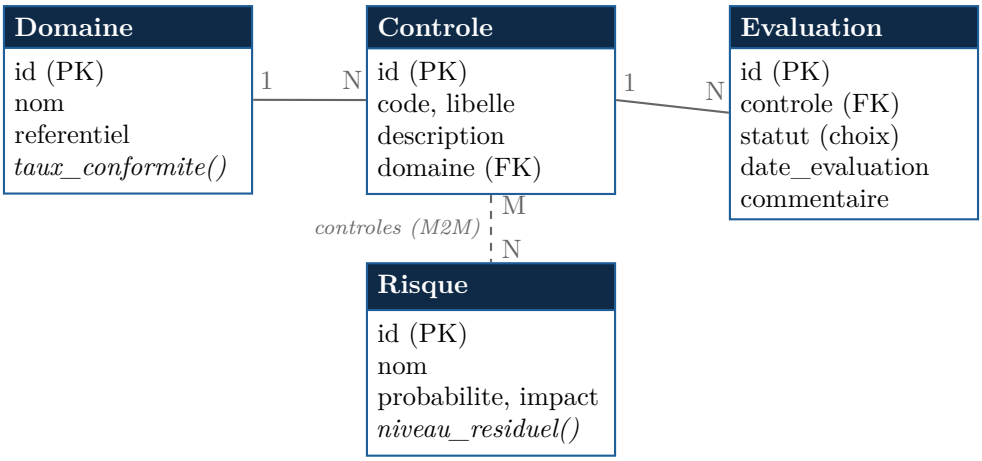


Fig. 2: Diagramme entité–association

Entité	Champs principaux	Remarques
Domaine	nom, referentiel	`taux_conformite()` renvoie le taux, ou `None` si rien n'est évalué
Controle	code, libelle, domaine	Clé étrangère vers Domaine, suppression en cascade
Evaluation	controle, statut, date, commentaire	statut $\in$ {conforme, partiel, non_conforme, na} ; date renseignée automatiquement
Risque	nom, probabilite, impact, controles	`niveau_residuel()` classe $P \times I$ en quatre niveaux

Tab. 5: Entités

Stocker les évaluations comme des enregistrements distincts, plutôt que comme un champ du contrôle, conserve une **piste d'audit** : un contrôle peut être réévalué dans le temps et chaque évaluation garde sa date et son commentaire.

6 Logique métier

6.1 Taux de conformité par domaine

Chaque statut est converti en points ; les contrôles *non applicables* sont exclus du numérateur et du dénominateur, comme dans une Déclaration d'Applicabilité (DdA) :

$$\text{Taux}(d) \;=\; \frac{\displaystyle\sum_{e \in E_d} \text{pts}(e)}{|E_d|} \times 100, \qquad \text{pts} = \begin{cases} 1 & \text{conforme} \\ 0{,}5 & \text{partiel} \\ 0 & \text{non conforme} \end{cases}$$

où E_d est l'ensemble des évaluations applicables du domaine d.

```python
def taux_conformite(self):
    points = {"conforme": 1, "partiel": 0.5, "non_conforme": 0}
    evaluations = Evaluation.objects.filter(
        controle__domaine=self
    ).exclude(statut="na")
    if not evaluations.exists():
        return None
    total_points = sum(points[e.statut] for e in evaluations)
    return round((total_points / evaluations.count()) * 100, 1)
```

Exemple de calcul — Organisationnel

A.5.9 est *partiel* (0,5) et A.5.19 est *non conforme* (0) : Taux $= \frac{0{,}5+0}{2} \times 100 = 25{,}0\,\%$. Renvoyer `None` lorsque rien n'est applicable distingue « pas encore évalué » de « conforme à 0 % ».

6.2 Cotation des risques

Chaque risque est coté sur deux échelles de 1 à 5. Le score $S = P \times I \in [1, 25]$ est classé comme suit :

```
def niveau_residuel(self):
    score = self.probabilite * self.impact
    if score >= 15: return "Critique"
    elif score >= 9: return "Élevé"
    elif score >= 4: return "Moyen"
    return "Faible"
```

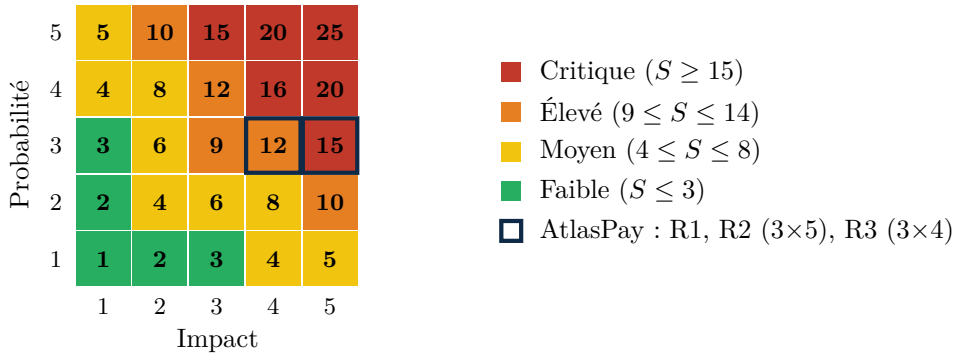


Fig. 3: Matrice des risques 5×5 ; les cases encadrées situent les risques d'AtlasPay

7 Réalisation pas à pas

Cette section retrace la construction du cas AtlasPay à l'aide des captures d'écran réalisées pendant le projet.

7.1 Back-office d'administration

Les quatre modèles sont enregistrés dans l'admin Django, qui fournit sans code supplémentaire des écrans authentifiés de création, consultation, modification et suppression.

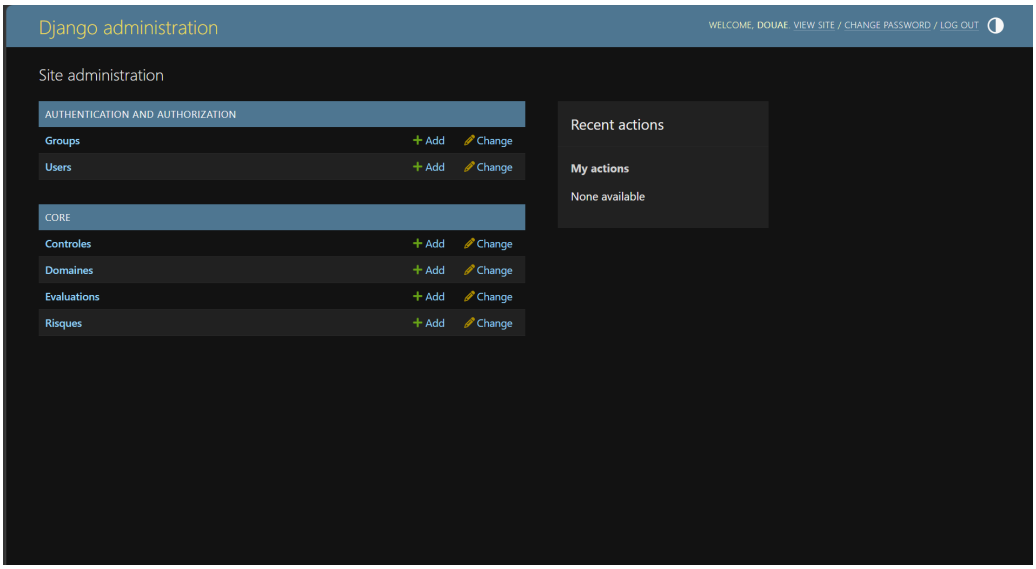


Fig. 4: Admin Django avec l'application `core` (Controles, Domaines, Evaluations, Risques)

7.2 Domaines et contrôles

Les cinq domaines ont été créés en premier, puis les huit contrôles, chacun rattaché à son domaine.

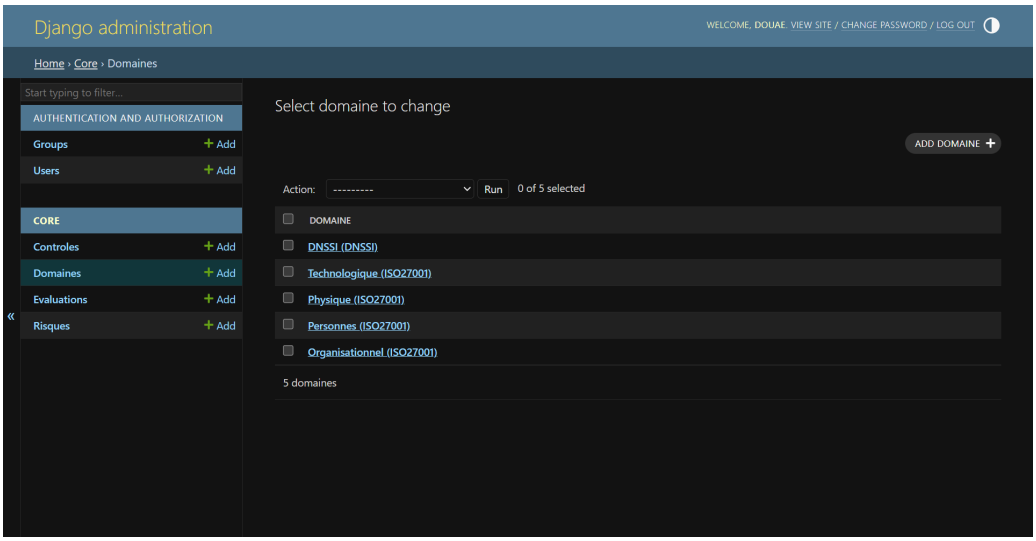


Fig. 5: Les cinq domaines : quatre thèmes ISO 27001 et la DNSSI

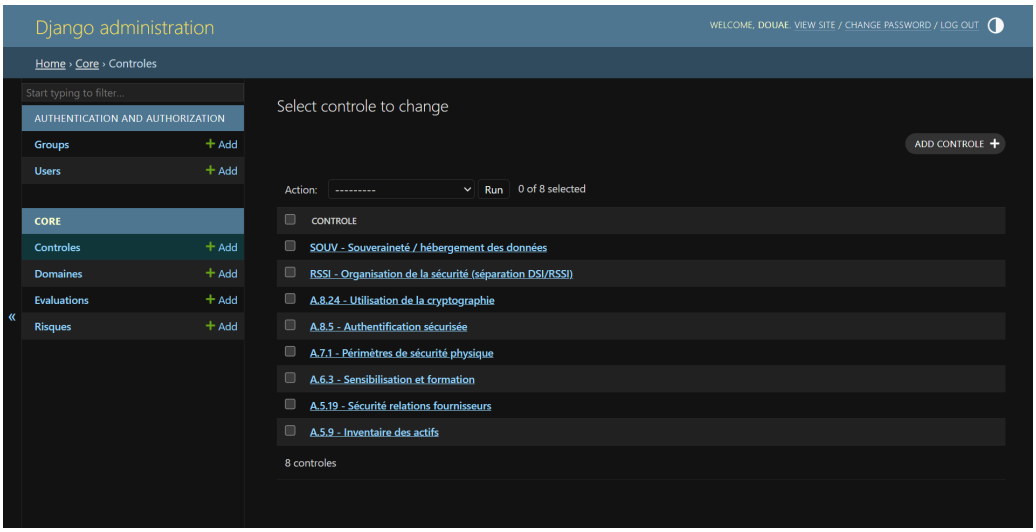


Fig. 6: Les huit contrôles du catalogue AtlasPay

7.3 Évaluations

Une évaluation a été saisie par contrôle, avec son statut et un commentaire facultatif pour les preuves.

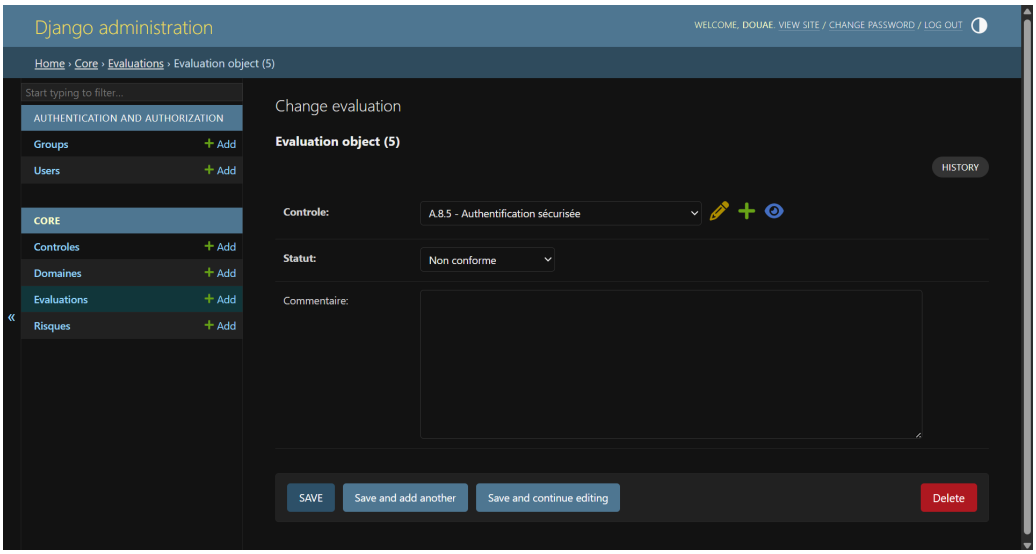


Fig. 7: Évaluation d'A.8.5 Authentification sécurisée : *Non conforme*

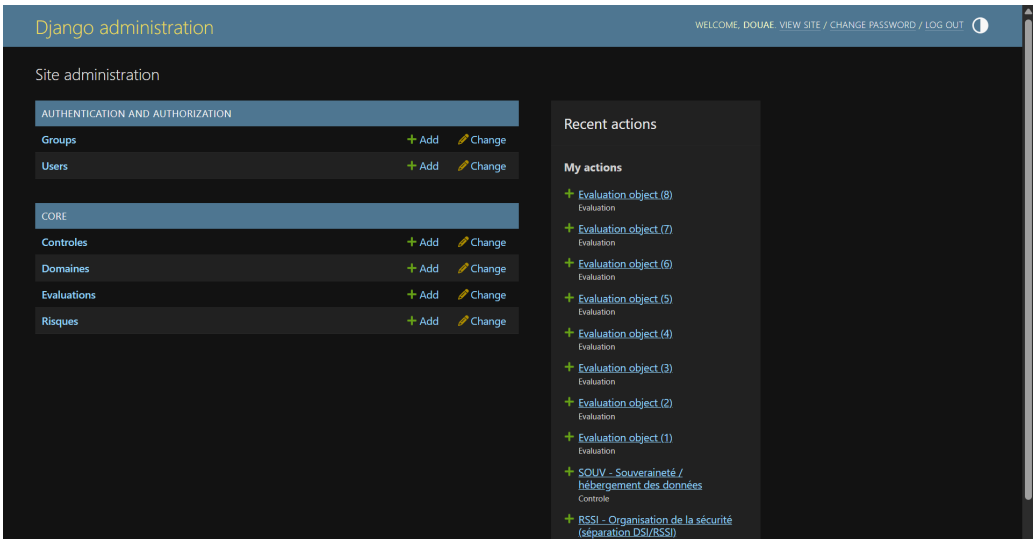


Fig. 8: Journal de l'admin montrant les contrôles et les huit évaluations créés

7.4 Vérification dans le shell Django

Avant de construire le dashboard, le calcul de conformité a été vérifié directement sur la base de données :

```
[28/Jul/2026 00:02:10] "GET /static/admin/css/dashboard.css HTTP/1.1" 304 0
% (venv) PS C:\Users\douae\grc_project> python manage.py shell
 16 objects imported automatically (use -v 2 for details).

 Ctrl click to launch VS Code Native REPL
 Python 3.13.0 (tags/v3.13.0:60403a5, Oct  7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
 Type "help", "copyright", "credits" or "license" for more information.
 (InteractiveConsole)
 >>> from core.models import Domaine
 >>>
 >>> for d in Domaine.objects.all():
 ...     print(d.nom, "->", d.taux_conformite(), "%")
 ...
 Organisationnel -> 25.0 %
 Personnes -> 50.0 %
 Physique -> 50.0 %
 Technologique -> 0.0 %
 DNSSI -> 25.0 %
 >>>
```

Fig. 9: Taux de conformité par domaine calculé par `taux_conformite()`

7.5 Dashboard

Le dashboard réunit les deux vues sur une seule page. Dans sa version finale, chaque page étend `base.html`, qui ajoute une barre de navigation, et le dashboard applique un code couleur : les taux sont classés `taux-bas` ($< 30\,\%$), `taux-moyen` ($< 70\,\%$) ou `taux-haut`, et chaque risque prend la classe de son niveau (`critique`, `élevé`, …).

Dashboard GRC — AtlasPay

Taux de conformité par domaine

Domaine	Référentiel	Taux de conformité
Organisationnel	ISO27001	25.0%
Personnes	ISO27001	50.0%
Physique	ISO27001	50.0%
Technologique	ISO27001	0.0%
DNSSI	DNSSI	25.0%

Registre des risques

Risque	Probabilité	Impact	Niveau résiduel
Fraude via API mal sécurisée	3	5	Critique
Ransomware sur les serveurs on-premise	3	5	Critique
Fuite de données clients	3	4	Élevé

Fig. 10: Dashboard GRC d'AtlasPay (première version, avant le gabarit commun et la feuille de style)

8 Résultats — évaluation d'AtlasPay

8.1 Conformité par domaine

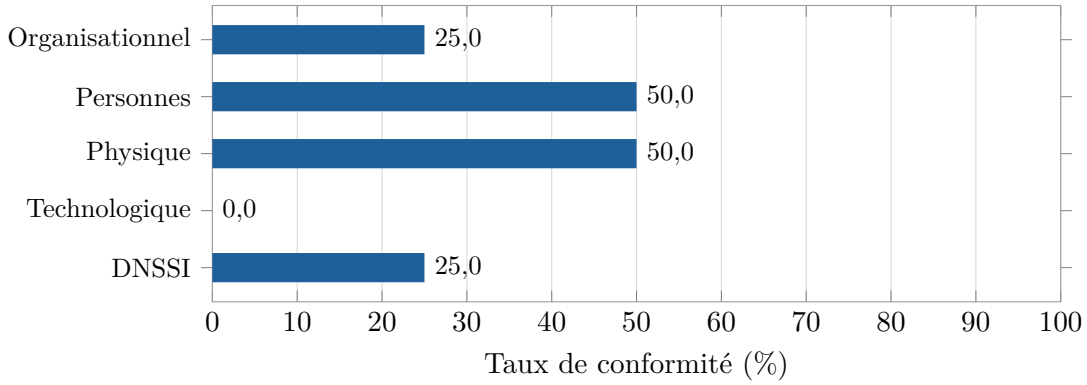


Fig. 11: Taux de conformité par domaine

Domaine	Contrôle	Statut	Date	Taux
Organisationnel	A.5.9	partiel	27/07/2026	25,0 %
	A.5.19	non conforme	27/07/2026	
Personnes	A.6.3	partiel	27/07/2026	50,0 %
Physique	A.7.1	partiel	27/07/2026	50,0 %
Technologique	A.8.5	non conforme	28/07/2026	0,0 %
	A.8.24	non conforme	28/07/2026	
DNSSI	RSSI	partiel	28/07/2026	25,0 %
	SOUV	non conforme	28/07/2026	

Tab. 6: Détail des évaluations (issu de la base de données du projet)

8.2 Registre des risques

N°	Risque	P	I	S	Niveau	Contrôles liés
R1	Fraude via API mal sécurisée	3	5	15	**Critique**	A.8.5
R2	Ransomware sur les serveurs on-premise	3	5	15	**Critique**	A.7.1, A.8.24
R3	Fuite de données clients	3	4	12	**Élevé**	A.5.19

Tab. 7: Registre des risques et correspondance risques–contrôles

8.3 Analyse

- **Aucun risque n'est couvert par un contrôle conforme.** R1 dépend d'A.8.5 (non conforme), R2 d'A.7.1 (partiel) et d'A.8.24 (non conforme), R3 d'A.5.19 (non conforme). Le registre et les évaluations racontent la même histoire.
- **La technologie est l'écart critique.** À 0 %, le domaine Technologique porte les contrôles des deux risques critiques : authentification forte sur les API de paiement (MFA, gestion des jetons) et chiffrement des données au repos et en transit.
- **La gouvernance est fragile.** Avec 25 %, le domaine Organisationnel révèle un inventaire des actifs incomplet et l'absence de sécurité formalisée avec les fournisseurs — seul contrôle relié au risque de fuite de données clients.

- **Exposition réglementaire DNSSI.** Une séparation DSI/RSSI partielle et un hébergement des données non conforme constituent un écart réglementaire direct pour un acteur marocain du paiement.

Ordre de remédiation recommandé

1. A.8.5 Authentification sécurisée — réduit R1 (critique).
2. A.8.24 Cryptographie — réduit R2 (critique) et protège aussi les données concernées par R3.
3. A.5.19 Sécurité fournisseurs — seul contrôle relié à R3.
4. A.7.1 Périmètres physiques — complète la couverture de R2 sur les serveurs internes.
5. SOUV et RSSI — comblent les écarts réglementaires DNSSI.

9 Tests et validation

Une suite de tests automatisés (`core/tests.py`) rend les règles de calcul vérifiables. Elle s'exécute avec `python manage.py test` et passe 8 tests sur 8.

Test	Vérifie que
`test_sans_evaluation_retourne_none`	Sans évaluation ⇒ `None`, pas 0 %
`test_ponderation`	$(1 + 0{,}5 + 0)/3 = 50\,\%$
`test_non_applicable_exclu`	N/A est exclu du dénominateur
`test_seulement_na_retourne_none`	Uniquement N/A ⇒ `None`
`test_seuils`	Tous les seuils de niveau de risque
`test_pages_repondent`	Les quatre pages répondent HTTP 200
`test_ajout_risque_redirige_vers_dashboard`	Un risque valide est enregistré puis redirigé
`test_dashboard_reproduit_les_taux_documentes`	Les données de démo reproduisent les taux documentés

Tab. 8: Suite de tests

La commande `python manage.py seed_demo` recrée le jeu de données AtlasPay à partir de la base d'origine (statuts et liens avec les risques), afin que toute personne clonant le dépôt obtienne le même dashboard. Elle est idempotente et accepte l'option `--reset`.

10 Revue de sécurité et de qualité

S'agissant d'un outil de sécurité, l'application doit elle-même respecter les bonnes pratiques. Le code et la base de données du projet ont été revus avant publication.

N°	Constat	Gravité	Statut
S1	`SECRET_KEY` écrite en dur dans les paramètres	Élevée	Corrigé — variable d'env.
S2	`DEBUG = True` écrit en dur	Moyenne	Corrigé — variable d'env.
S3	`ALLOWED_HOSTS` non paramétrable	Faible	Corrigé — variable d'env.
S4	Base de données (hash du mot de passe admin, e-mail) dans le dossier du projet	Moyenne	Corrigé — `.gitignore`
S5	Pages applicatives accessibles sans authentification	Moyenne	Ouvert — feuille de route
S6	`probabilite/impact` acceptent n'importe quel entier	Faible	Ouvert — feuille de route
Q1	Contrôle A.8.24 en double dans la base (second enregistrement sans évaluation, relié à R2)	Faible	Ouvert — feuille de route

Tab. 9: Constats

Le constat Q1 montre l'intérêt d'une contrainte d'unicité sur `Controle.code` : le doublon n'a pas d'évaluation et n'entre donc pas dans le taux de conformité, mais il apparaît comme contrôle de traitement de R2.

Protections déjà actives grâce au framework : jetons CSRF sur chaque formulaire, protection contre le clickjacking (`X-Frame-Options: DENY`), requêtes paramétrées par l'ORM (pas de surface d'injection SQL dans le code applicatif), échappement automatique des templates (XSS) et validateurs de mots de passe pour les comptes admin.

11 Limites et feuille de route

Limite	Amélioration prévue
Le « niveau résiduel » n'utilise que $P \times I$: c'est en réalité le risque *inhérent*	Réduire le score selon la conformité des contrôles liés
Pas de contrôle d'accès sur les vues applicatives	`@login_required`, rôles (auditeur, propriétaire du risque, lecture seule)
Probabilité et impact non bornés	`MinValueValidator(1)`, `MaxValueValidator(5)`
Codes de contrôle en double possibles	`unique=True` sur `Controle.code`
Le taux compte toutes les évaluations, pas la dernière par contrôle	Dernière évaluation par contrôle, historique et tendances
Un taux `None` s'affiche « None% » sur le dashboard	Afficher « N/A » pour les domaines non évalués
Évaluations affichées « Evaluation object (n) » dans l'admin	`__str__`, `list_display`, filtres
Catalogue partiel (8 contrôles)	Annexe A complète (93 contrôles) et import DNSSI

Tab. 10: Limites et feuille de route

12 Guide d'installation

Prérequis : Python 3.12 ou plus récent et Git.

```
git clone https://github.com/Dorvae/grc-compliance-tool.git
cd grc-compliance-tool
python -m venv venv
venv\Scripts\activate          # Linux/macOS : source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py seed_demo       # données de démonstration (optionnel)
python manage.py createsuperuser
python manage.py runserver
```

Le dashboard est alors disponible à l'adresse `http://127.0.0.1:8000/dashboard/` et le back-office à `/admin/`. En dehors du développement local, renseigner les variables de `.env.example` avec une nouvelle clé secrète aléatoire et `DJANGO_DEBUG=False`.

13 Conclusion

Le projet livre un outil GRC fonctionnel qui transforme une liste de contrôles de sécurité en indicateurs de conformité mesurables et les relie aux risques métier. Appliqué à AtlasPay, il montre sur une seule page que les contrôles technologiques constituent le point faible et qu'ils sous-tendent les deux risques critiques — exactement le type de priorisation fondée sur des preuves dont les équipes d'audit et de risques ont besoin. Le code est compact, testé et structuré pour que les éléments de la feuille de route puissent être ajoutés progressivement.

A Structure du dépôt

```
grc-compliance-tool/
|-- README.md  LICENSE  requirements.txt  .env.example  .gitignore
|-- manage.py
|-- grc_project/   settings.py  urls.py  wsgi.py  asgi.py
|-- core/          models.py  views.py  forms.py  admin.py  tests.py
|   |-- templates/        base, home, dashboard, 2 formulaires
|   |-- static/core/      style.css
|   |-- migrations/       0001_initial.py
|   `-- management/commands/seed_demo.py
`-- docs/
    |-- GRC_Tool_Documentation.pdf
    |-- latex/            source LaTeX de ce document
    `-- screenshots/
```